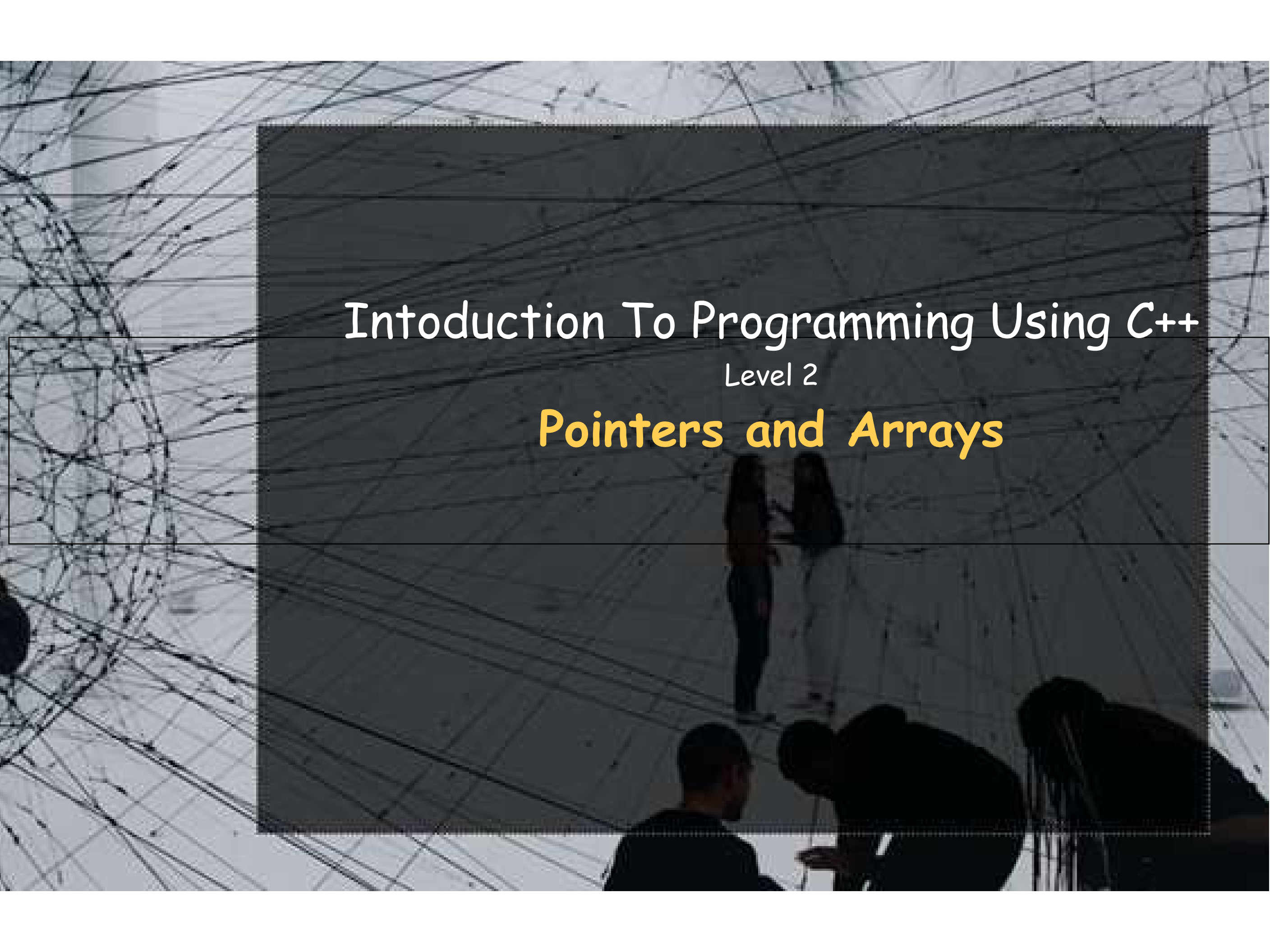


Introduction To Programming Using C++ Level 2

Pointers and Arrays



Introduction To Programming Using C++

Level 2

Pointers and Arrays

Definition

Array

 An Array is a collection of elements of the same type stored in contiguous memory locations.

 The array name can be used in most expressions as a pointer to the first element of the array.

 For an array named arr, the array name represents the address of the first element:

$\text{arr} \rightarrow \&\text{arr}[0]$

Explanation

Connecting an Array to a Pointer

 We can create a pointer and make it point to the first element of an array.

 Example:

```
int arr[4] = {10, 20, 30, 40};
```

```
int* ptr;
```

```
ptr = arr;
```

 Here, ptr points to arr[0].

Pointer Arithmetic

 When we increase ptr by 1, it moves to the next element of the array.

 It does not simply move to the next byte in memory.

 The movement is based on the size of the pointer's type.

Procedures

- 1 Create an array.
- 2 Store several values in the array.
- 3 Create a pointer of the same type as the array elements.
- 4 Assign the array name to the pointer:
 `ptr = arr;`
- 5 The pointer now points to the first element.
- 6 Use `ptr + 1` to move to the next element.
- 7 Use `*(ptr + 1)` to access the value of the next element.
- 8 Continue using pointer arithmetic to access other elements.

💡 Example

◆ Array and Pointer

```
int arr[4] = {10, 20, 30, 40};  
int* ptr;  
ptr = arr;  
cout << ptr << endl;  
cout << ptr + 1 << endl;  
cout << *ptr << endl;  
cout << *(ptr + 1) << endl;
```

➡ Result

```
ptr → Address of arr[0]  
ptr + 1 → Address of arr[1]  
*ptr → 10  
*(ptr + 1) → 20
```

Note

- ◆ Pointer arithmetic is based on the type of the pointer.
- ◆ For an int pointer:
 `int* ptr;`
  `ptr + 1` moves to the next int element.
- ◆ For a different pointer type, the movement is based on the size of that type.
- ◆ The exact number of bytes depends on the size of the pointed-to type on the system.

⊘ Warning

◆ The array name is not itself a modifiable pointer variable.

◆ You can do:

```
ptr = arr;
```

```
ptr++;
```

◆ But you cannot do:

```
arr++;
```

◆ The array name represents the array itself, and in most expressions it converts to a pointer to its first element.



Comparison

▨ Array Name

- ✓ In most expressions, represents the address of the first element.
- ✓ Example:
 `arr → &arr[0]`
- ✓ Cannot be incremented like a pointer variable.

▨ Pointer

- ✓ Is a separate variable that stores an address.
- ✓ Can point to the first element of an array.
- ✓ Can be incremented or decremented to move between elements.

✦✦ Summary

- ✓ An Array stores elements of the same type in contiguous memory locations.
- ✓ The array name usually represents the address of the first element.
- ✓ $\text{arr} \rightarrow \&\text{arr}[0]$
- ✓ $\text{ptr} = \text{arr} \rightarrow$ The pointer points to the first element.
- ✓ $\text{ptr} + 1 \rightarrow$ Points to the next array element.
- ✓ $*\text{ptr} \rightarrow$ Accesses the value of the current element.
- ✓ $*(\text{ptr} + 1) \rightarrow$ Accesses the value of the next element.
- ✓ Pointer arithmetic moves according to the size of the pointed-to type.

Thank you 😊